Repositorio e inyección de dependencias manual

Mars Photos · Axel Adrián Gaviña González
28 de octubre de 2026

1. Objetivo y alcance

Separar el acceso a datos de la interfaz de Mars Photos y suministrar las dependencias desde un contenedor de aplicación. Se implementó la solución sobre el proyecto inicial oficial y se verificó su comportamiento con pruebas ejecutadas en la PC.

La app consulta una lista de fotografías y presenta la cantidad recibida. El alcance de esta práctica es el contador y la arquitectura de datos; no corresponde todavía a la galería de fotografías.

Código e historial: Axel421-yn/mars-photos-repositorio-di

2. Entorno de trabajo

Componente	Configuración utilizada
Sistema	Windows; ejecución local de Gradle y JUnit
Lenguaje e interfaz	Kotlin 1.9.10 · Jetpack Compose · Material 3
Herramientas	JDK 21.0.8 · Gradle 8.11.1 · AGP 8.9.2
Android	compileSdk 35 · targetSdk 34 · minSdk 24
Pruebas	JUnit 4.13.2 · coroutines-test 1.7.3 · Robolectric 4.14.1

Se ajustaron Gradle, AGP y compileSdk para utilizar el JDK y el SDK disponibles. Se conservaron las dependencias principales y el comportamiento de la base del curso.

3. Punto de partida

Se utilizó la rama repo-starter, commit ba9470a, del repositorio de formación de Google. El código original mantiene su licencia Apache 2.0. PROCEDENCIA.md identifica la fuente y los commits de GitHub documentan las etapas de la solución.

Arquitectura y flujo de datos

4. Responsabilidades

Elemento	Responsabilidad en la solución
MarsApiService	Declara GET /photos y devuelve objetos MarsPhoto mediante una función suspendida.
MarsPhotosRepository	Define el contrato de acceso a los datos sin exponer Retrofit a la interfaz.
NetworkMarsPhotosRepository	Recibe MarsApiService por constructor y delega la consulta al servicio.
DefaultAppContainer	Configura Retrofit y expone una única instancia diferida del repositorio.
MarsPhotosApplication	Inicializa el contenedor en onCreate; se registra en AndroidManifest.xml.
MarsViewModel.Factory	Obtiene el contenedor desde Application e inyecta el repositorio al ViewModel.
MarsPhotosApp / HomeScreen	Observan MarsUiState y muestran carga, éxito o error.

5. Recorrido de una consulta

Interfaz -> ViewModel -> repositorio -> servicio Retrofit -> API.
La respuesta retorna como una lista; el ViewModel calcula su tamaño y cambia el estado visible. IOException y HttpException producen el estado Error.

La creación de objetos queda concentrada en el contenedor. El ViewModel recibe una interfaz y no construye el servicio de red. Para las pruebas se reemplaza el repositorio por una implementación controlada sin modificar el código de producción.

6. Pruebas reproducibles

TestDispatcherRule instala un despachador de corrutinas para el hilo principal durante cada prueba y lo restaura al terminar. FakeDataSource contiene dos fotos; FakeMarsApiService y FakeNetworkMarsPhotosRepository entregan esos datos sin acceso a Internet.

Las pruebas visuales ejecutan MarsPhotosApp y su Factory real con un contenedor de prueba. Las imágenes se exportan desde el árbol Compose con renderizado nativo de Robolectric. Esto comprueba la interfaz en la JVM local, pero no sustituye una prueba en hardware Android.

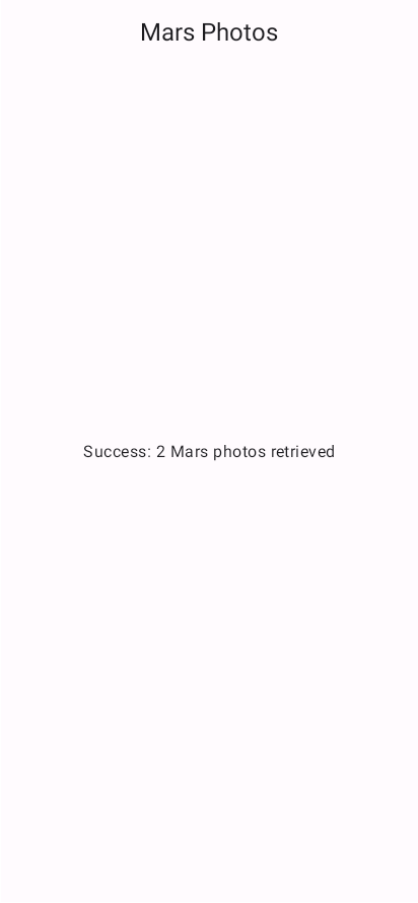
Evidencias de ejecución en la PC

7. Compilación, pruebas y análisis

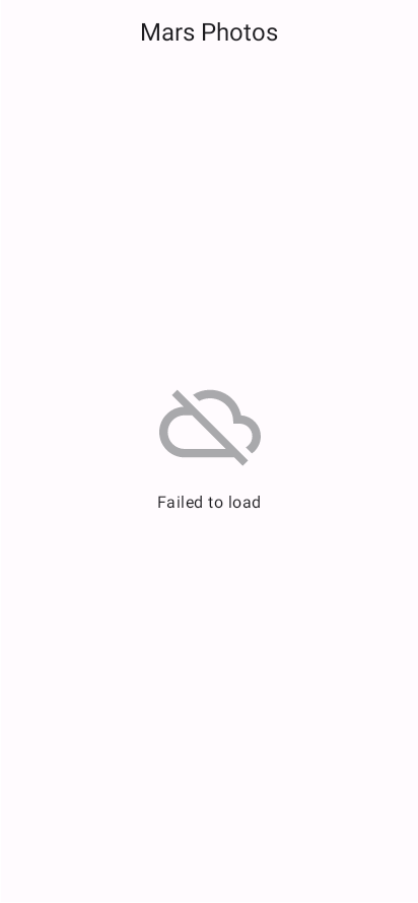
Validación	Resultado observado
assembleDebug	BUILD SUCCESSFUL; APK generado.
testDebugUnitTest	10 pruebas ejecutadas; 0 fallos/errores.
lintDebug	0 errores; 17 advertencias. Diagnóstico completo en lint.txt.
Consulta externa desde PowerShell	HTTP 200; 838 registros recibidos el 28/10/2026. Esta consulta es independiente de las pruebas con datos falsos.

Cobertura funcional

Repositorio: lista recibida, lista vacía y propagación de IOException.
ViewModel: éxito, carga pendiente, error de E/S, error HTTP y recuperación posterior.
Interfaz: resultado con dos registros inyectados y mensaje ante fallo de red.



Éxito: dos registros de prueba



Error: fallo de red de prueba

Capturas obtenidas al ejecutar Compose en Robolectric/API 33, JVM local. No son capturas de un teléfono ni del emulador Android.

Reproducción, límites y conclusiones

8. Cómo reproducir

Clonar el repositorio y abrirlo en Android Studio. Seleccionar JDK 21, instalar SDK 35 y sincronizar Gradle. Ejecutar estos comandos desde la raíz del proyecto en Windows:

```
.\gradlew.bat assembleDebug
.\gradlew.bat testDebugUnitTest lintDebug
```

El APK se genera en app/build/outputs/apk/debug/app-debug.apk. Los resultados HTML se generan en app/build/reports/tests/testDebugUnitTest/. Las capturas de las pruebas visuales se escriben en app/build/evidence/.

Para probar en Android, seleccionar un dispositivo API 24 o superior con Internet y ejecutar app. Debe aparecer primero la carga y después el contador de registros. Si la consulta falla, se muestra Failed to load.

9. Límites de la evidencia

El emulador Medium_Phone no pudo arrancar con aceleración de hardware en el entorno disponible; el intento por software permaneció sin conexión. Por ello no se afirma haber instalado ni ejecutado el APK en un emulador o teléfono. La compilación, JUnit, el renderizado local de Compose y la consulta HTTP son evidencias distintas y se identifican por separado.

Lint reportó advertencias relacionadas principalmente con versiones de dependencias, targetSdk y recursos heredados sin uso. No impidieron la compilación. Se conserva el diagnóstico completo en docs/evidencias/lint.txt.

10. Conclusiones

La separación del repositorio eliminó la llamada directa al servicio desde el ViewModel. La DI manual permitió cambiar el origen de datos por objetos de prueba y verificar estados de éxito, carga y error de forma controlada. La Factory conecta esa arquitectura con el ciclo de vida de Compose.

Los resultados respaldan la lógica y la integración local de la interfaz. Una validación posterior en un dispositivo Android complementaría la evidencia con el comportamiento real del sistema, conectividad y ciclo de vida. La solución conserva una estructura sencilla y permite añadir otras fuentes de datos sin reescribir la interfaz.

Fuentes y anexos

Android Developers: Cómo agregar un repositorio y una DI manual
Google: código inicial de Mars Photos
Historial de la solución en GitHub

Anexos: logs de Gradle, XML de JUnit, resultado de Lint, consulta-api.json, capturas PNG y APK compilado. Los anexos permiten comprobar los resultados sin depender únicamente del texto de este informe.